

# Week 2 - Day 9b: Debugging and Comprehensions

In today's notebook, we will first cover how to understand and tackle **errors**. This is very important as our code often doesn't work on the first try! Understanding what's wrong and what to do next is crucial for us to debug.

For the second half of today's exercise, we will practice **string manipulation** and **list and dictionary comprehensions**, which are very useful ways to process data structures in a **very clean style**!

## Exercise 1: Understanding Errors [ ]

Oops! Each snippet of the following code crashes. Run the code, **scroll to the bottom** of the error message, and read it. See if you can find the line of code where the error occurred by reading the messages!

Try to fix the errors! **DO NOT** copy your code from the previous labs, but make **minor** changes to the code here.

**1.1** The function `fibonacci` tries to compute the  $n$ -th Fibonacci number but something is wrong. Fix the function.

```
# TODO: Fix the function
def fibonacci(n):
    if n <= 1:
        return 1
    else:
        fibonacci(n-1) + fibonacci(n-2)
```

**1.2** The function `exp` attempts to compute  $a^b$  but something is wrong. Fix the function.

```
# TODO: Fix the function
def exp(a, b):
    return a*exp(a, b-1)
```

**1.3** The function `reverse_string` tries to turn a string backwards but something is wrong. Fix the function.

```
# TODO: Fix the function
def reverse_string(s):
    if len(s) == 0:
        print('')
    print(s[-1] + reverse_string(s[:-1]))
```

1.4 The function `factorial` tries to compute factorials but something is wrong. Fix the function.

```
# TODO: Fix the function
def factorial(n):
    if n == 1:
        return 1
    return n * factorial(n)
```

1.5 The function `sum_lst` tries to sum all numbers in a list but something is wrong. Fix the function.

```
# TODO: Fix the function
def sum_lst(lst):
    if len(lst) == 0:
        return 0
    else:
        lst[0] + sum_lst(lst[1:])
```

1.6 The function `catalan` tries to computing the  $n$ -th Catalan number but something is wrong. Fix the function. Remember, that the Catalan number sequence is defined as

$$C_0=1$$

$$C_n = \sum_{i=0}^{n-1} C_i \cdot C_{n-1-i}$$

```
# TODO: Fix the function
def catalan(n):
    if n == 0:
        return 1
    else:
        summ = 0
        for i in range(n):
            summ += catalan(i)*catalan(n-1-i)
        return summ
```

1.7 The function `permutations` tries to compute all permutations of numbers between 0 and  $n$  but something is going wrong. Fix the function.

```
# TODO: Fix the function
def permutations(n):
    if n == 0:
        return [[0]]
    else:
        new_permutations = []
        for i in range(n+1):
            for per in permutations(n-1):
```

```

        new_permutation = per[:i] + [n] + per[i:]
        new_permutations.append(new_permutation)
    return new_permutations

```

## Exercise 2: F-Strings [ ]

**F-Strings**, also called formatted **string literals**, allow you to format strings in a more readable and concise way. Normally strings in Python are enclosed within double quotes (") or single quotes ('). To create f-strings, you only need to add an **f** before the opening quotes of your string. For example, "Here" is a string where as **f"Here"** is an F-string.

When using f-Strings to display variables, you only need to specify the names of the variables inside a set of curly braces {}. And at runtime, all variable names will be replaced with their respective values. Check out the cell below for an example.

```

# Try run this code
lang = "python"
program = "AddisCoder"

print(f"I am learning {lang} in {program}.")

```

**2.1** Write a function `print_sum` that accepts two numbers `a` and `b` as inputs and then prints the sum of the input numbers using an F-string in the format `The sum of {a} and {b} is {a+b}`.

```

print_sum(1, 2) # Should print: "The sum of 1 and 2 is 3"
print_sum(0, 0) # Should print: "The sum of 0 and 0 is 0"

# TODO: Your code here
def print_sum(a, b):
    pass

```

**1.2** Given the dictionary `favourite_foods` and a TA `ta`, use F-strings to print `Nick's favourite dish is Fondue`.

```

favourite_foods = {
    "Saba": "Injera with Doro Wat",
    "Param": "Biryani",
    "Mihret": "Tibs",
    "Bruk": "Kitfo",
    "Amanuel": "Masgouf",
    "Meron": "Doro Wat",
    "Bontu": "Shiro",
    "Natneam": "Misir Wat",
    "Biniyam_A": "Gored Gored",
    "Huacheng": "Koshari",
    "Naol": "Kitfo",
    "Daniel": "Injera with Doro Wat",
}

```

```

    "Senem": "Peking Duck",
    "Nils": "Meatballs with Lingonberry",
    "Betelhem": "Hilbet",
    "Biniyam_L": "Dulet",
    "Lwam": "Chicago-style Pizza",
    "Nardos": "Injera with Kitfo",
    "Alex": "Plov",
    "Matthew": "Caesar Salad",
    "Nick": "Fondue",
    "Amanuel": "Injera with Tibs",
    "Hana": "Injera with Doro Wat",
    "Litus": "Tibs",
    "Sonika": "Butter Chicken",
    "Abdishakur": "Suqaar",
}

ta = "Nick"

# TODO: Your code here

```

## Exercise 3: String Split and Join [ ]

The split method can be used to separate each word in a string. The syntax is like this:

```

txt = "welcome to the world of programming"
x = txt.split() # ["welcome", "to", "the", "world", "of",
"programming"]

```

The join method does the reverse operation. It can be used to join strings in a sequence, such as a list. The syntax is like this:

```

mylist = ["Mihret", "Bini", "Dawit", "Hana", "Makda"]
x = "_".join(mylist) # "Mihret_Bini_Dawit_Hana_Makda"

```

Both methods are really useful to manipulate strings.

**3.1** Define a variable `sentence` that stores a sentence of your choice. Now, split the words of the sentence using the split method.

```

# TODO: Your code here

```

**3.2** Given the following list join the names of the TAs using '\*' as a separator.

```

tas = ["Nardos", "Betti", "Sonika", "Lwam", "Matthew"]

# TODO: Your code here

```

## Exercise 4: List Comprehensions [[]]

A common programming pattern is iterate over a sequence of numbers, perform some computation and store the result in a list. In Python, there is a more concise way to do this using **list comprehensions**.

Let's look at an example. To create a new list that contains the absolute values of the numbers in the list `nums`, we can do the following:

```
nums = [-10, -5, 0, 5, 10]
abs_nums = []
for num in nums:
    abs_nums.append(abs(num))

print(abs_nums)

[10, 5, 0, 5, 10]
```

This works fine, but can we do this in *one line*? Yes, with **list comprehension**! The syntax for list comprehension is as follows:

```
[expression for item in iterable]
```

Let's convert the above code into a list comprehension:

```
abs_nums = [abs(num) for num in nums] # List comprehension
print(abs_nums)
```

**4.1** Convert the following code into a list comprehension.

```
lst = []
for i in range(10):
    lst += [i]

# TODO: Your code here
```

**4.2** Convert the following code into a list comprehension.

```
string = 'HelloWorld'
lst = []

for letter in string:
    lst.append(letter)

# TODO: Your code here
```

**4.3** Write a list comprehension that generates a list of squares of numbers from 0 to 10.

```
# TODO: Your code here
```

**4.4** Write a list comprehension that filters out only even numbers from the list `lst=[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`.

```
# TODO: Your code here
```

## Exercise 5: Dictionary Comprehensions [ ]

Similarly to list comprehension, we can process dictionary much more concisely using **dictionary comprehension**! This is useful as we build new dictionaries, or manipulate the keys/values of dictionary.

In the below example we are capitalizing the first letter of the keys and values in the dictionary using a for loop.

```
words = {'nick': 'apple', 'saba': 'banana', 'mo': 'pineapple'}
capitalized_words = {}

for key, value in words.items():
    capitalized_words[key.capitalize()] = value.capitalize()

print(capitalized_words)
```

We can do this much more concisely with dictionary comprehension. The syntax for dictionary comprehension is as follows:

```
{expression for item in iterable}

capitalized_words = {key: value.capitalize() for key, value in
words.items()} # Dictionary comprehension
print(capitalized_words)
```

**5.1** Convert the following code into a dictionary comprehension.

```
square_dict = {}
for num in range(1, 11):
    square_dict[num] = num*num

# TODO: Your code here
```

**5.2** Write a dictionary comprehension that convert a list of names into a dictionary where the names are keys and the length of their name are values using a dictionary comprehension.

```
# TODO: Your code here
names = ["Jelani", "Sonika", "Litus", "Joseph", "Meron"]
```

**5.3** Given a dictionary of students' name and their corresponding scores, create a new dictionary using dictionary comprehension with only students who passed (scored 60 or above).

```
# Given dictionary of students' names and scores
students_scores = {
    'Blen': 79,
    'Samuel': 55,
    'Ekram': 75,
    'Mathias': 92,
    'Daniel': 40
}

# TODO: Your code here
```

**5.4** Given a dictionary with temperatures in Celsius for different cities, use dictionary comprehension to convert the temperatures to Fahrenheit.

*Hint: The formula to convert Celsius to Fahrenheit is  $F = C \times \frac{9}{5} + 32$*

```
# Given dictionary with temperatures in Celsius for different cities
celsius_temperatures = {
    'Adama': 25,
    'Addis Ababa': 18,
    'Semera': 30,
    'Mekelle ': 28,
    'Hawassa': 35,
    'Gonder': 23
}

# TODO: Your code here
```

Congratulations 